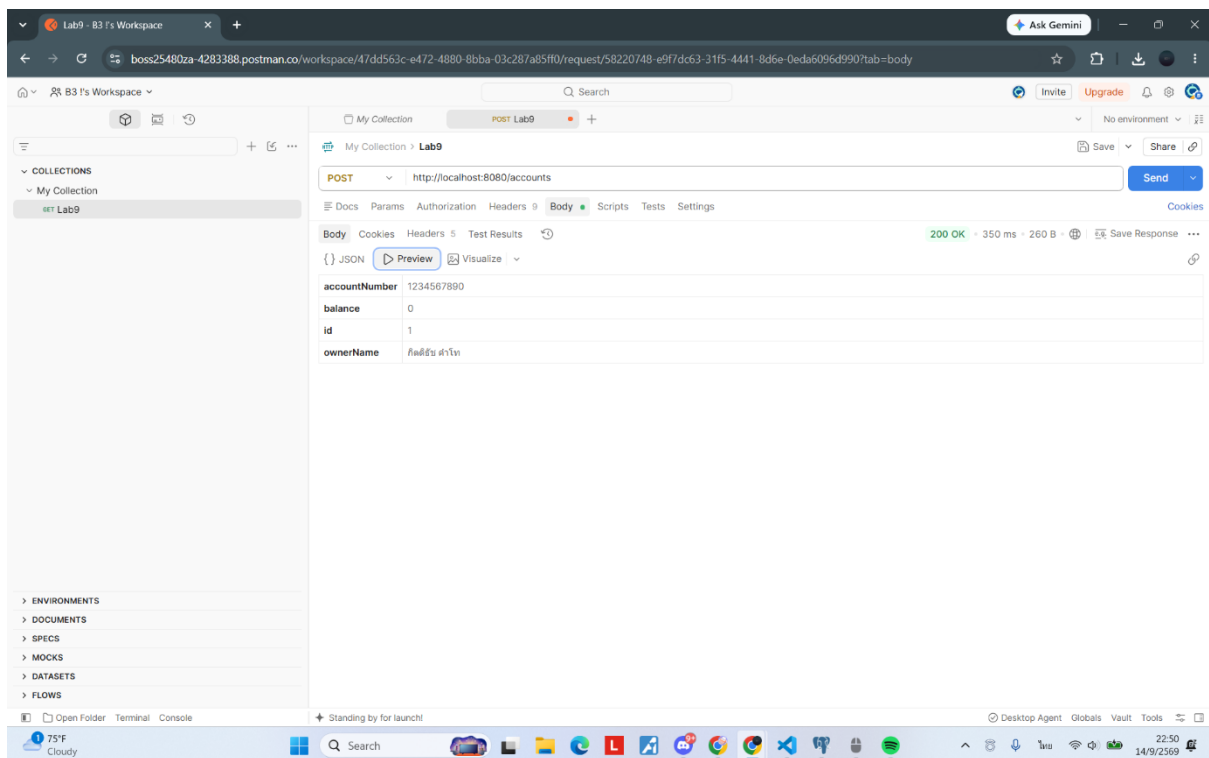


1. การสร้าง Account



ส่ง POST ไป /accounts

พร้อมข้อมูล{

"accountNumber": "1234567890",

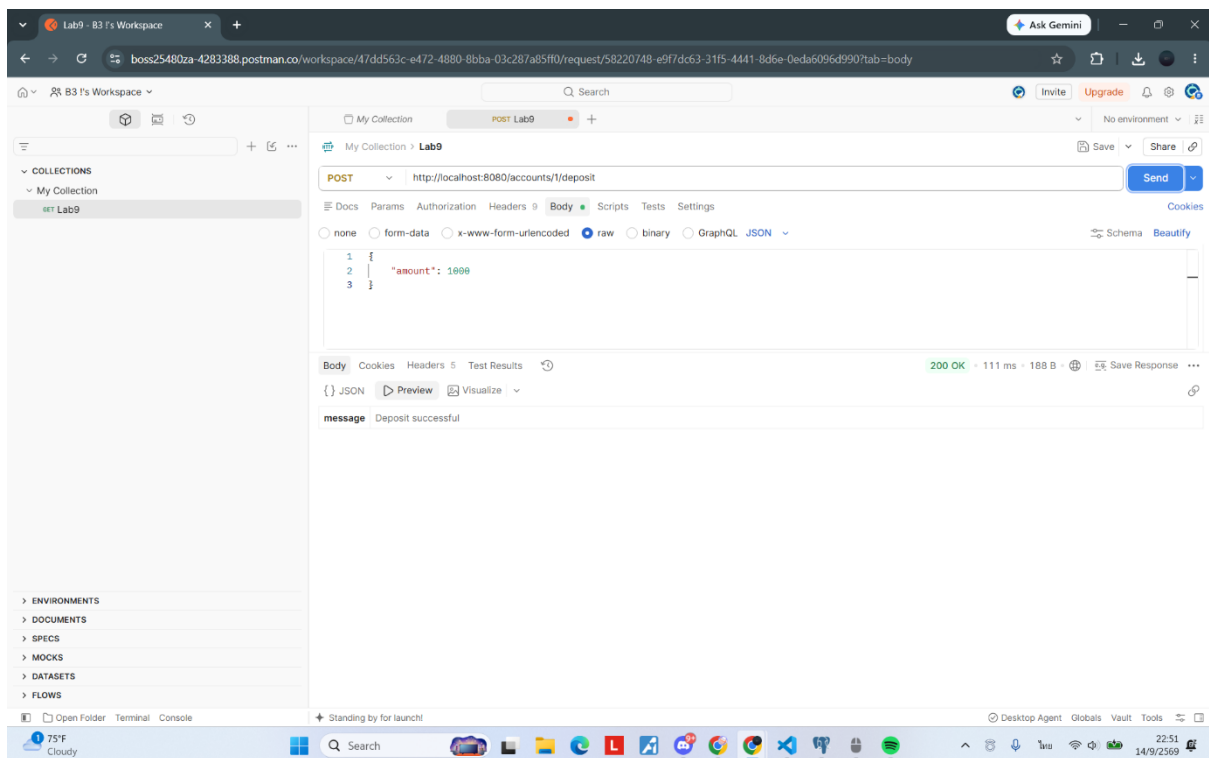
"ownerName": "ใส่ชื่อตัวเองนะ",

"balance": 0

}

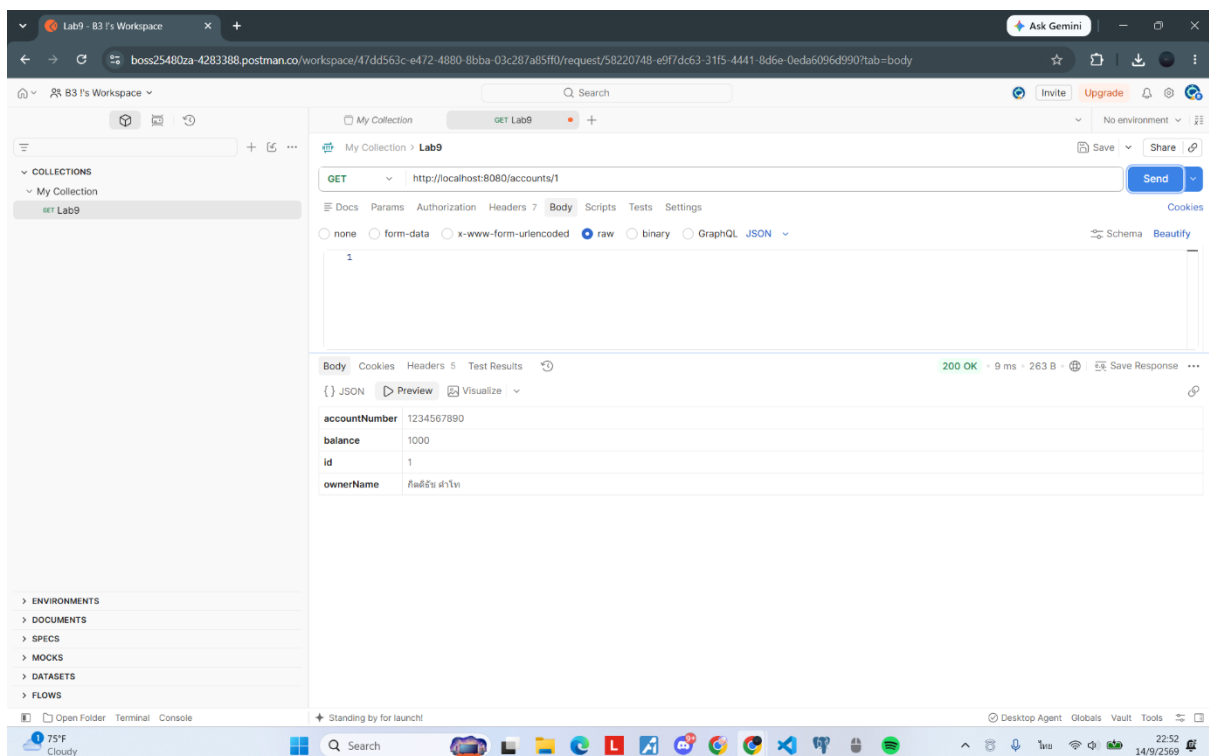
ได้ Status คือ 200 และ Account ที่สร้างได้ id 1

2. การฝากเงินสำเร็จ



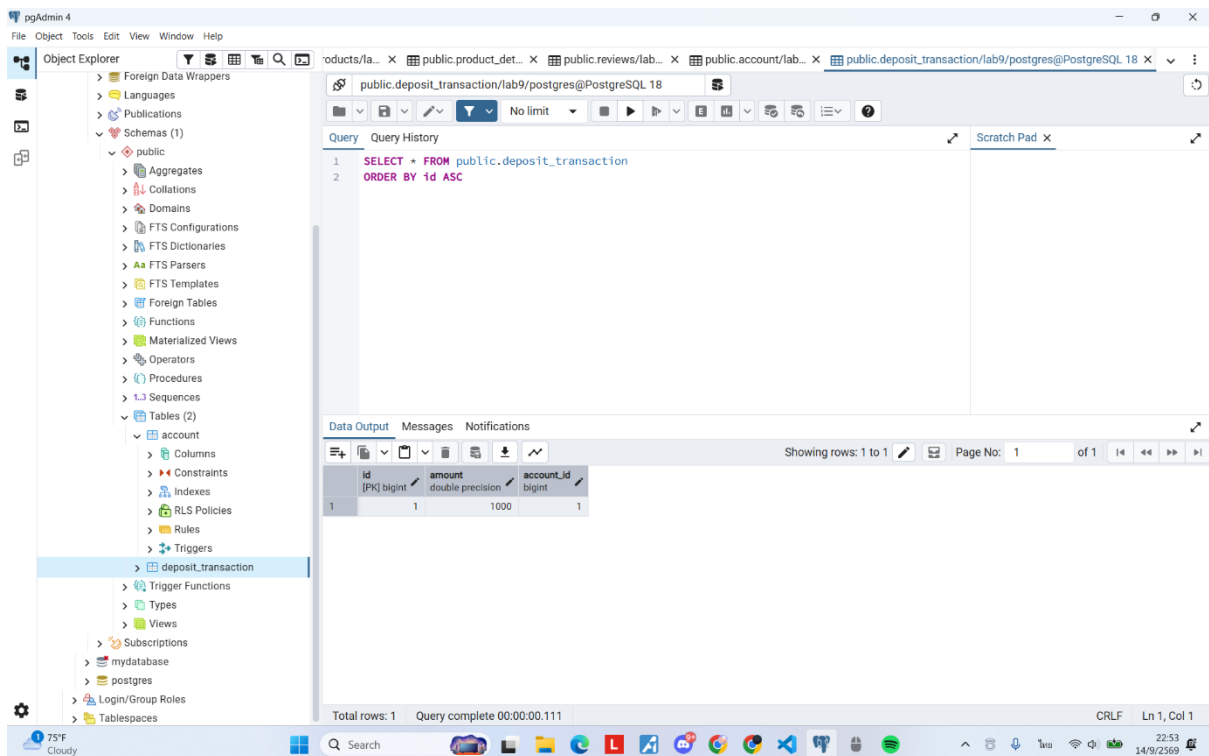
ส่ง POST ไป `/accounts/{id}/deposit` ด้วยจำนวนเงิน 1000 ได้ Status และ response message เป็น 200 OK + "Deposit successful"

3. ตรวจสอบ Account



ส่ง GET ไป /accounts/{id} แล้ว balance เปลี่ยนจาก 0 เป็น 1000 ตรงกับจำนวนที่ฝากไปในข้อ 2

4. ตรวจสอบ DepositTransaction



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including the 'public' schema and the 'deposit_transaction' table. The main pane shows a SQL query:

```
1 SELECT * FROM public.deposit_transaction
2 ORDER BY id ASC
```

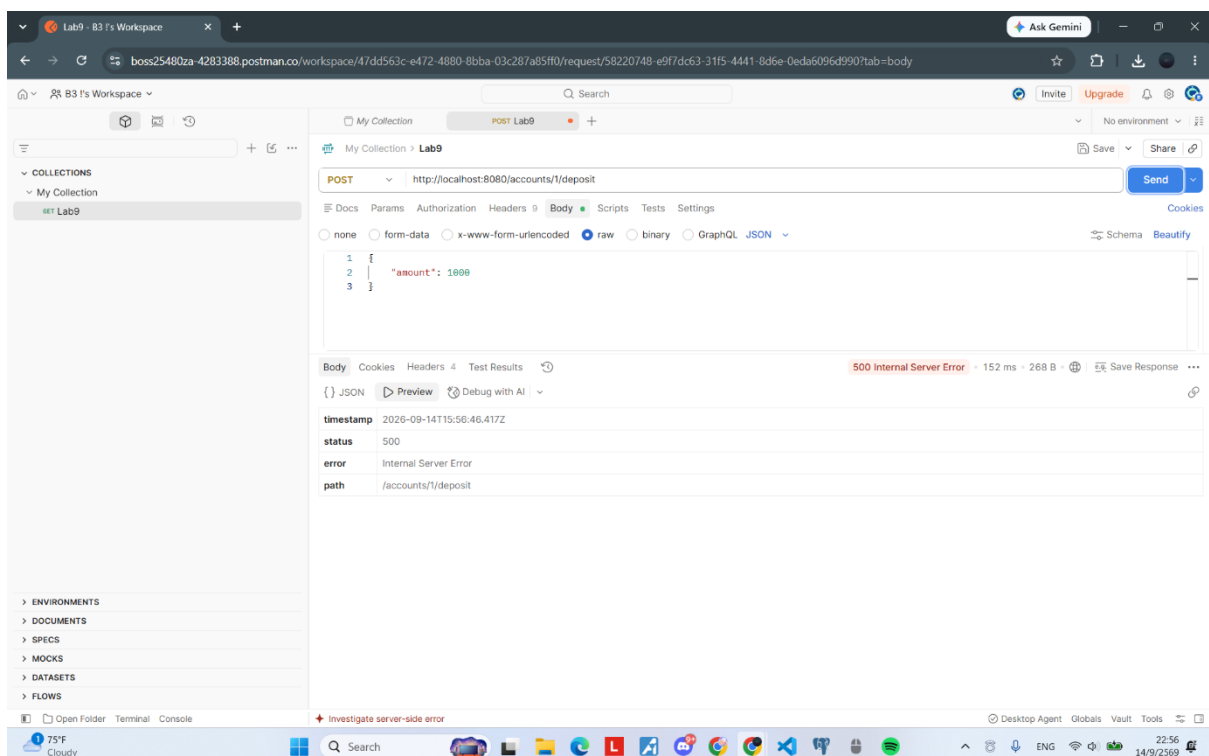
The Data Output pane shows the results of the query:

id	amount	account_id
1	1000	1

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.111'.

จากตาราง deposit_transaction ใน pgAdmin มี 1 แถว แต่ละแถวมี amount และ account_id ตรงกับที่ฝากไป

5. ทดลอง Rollback



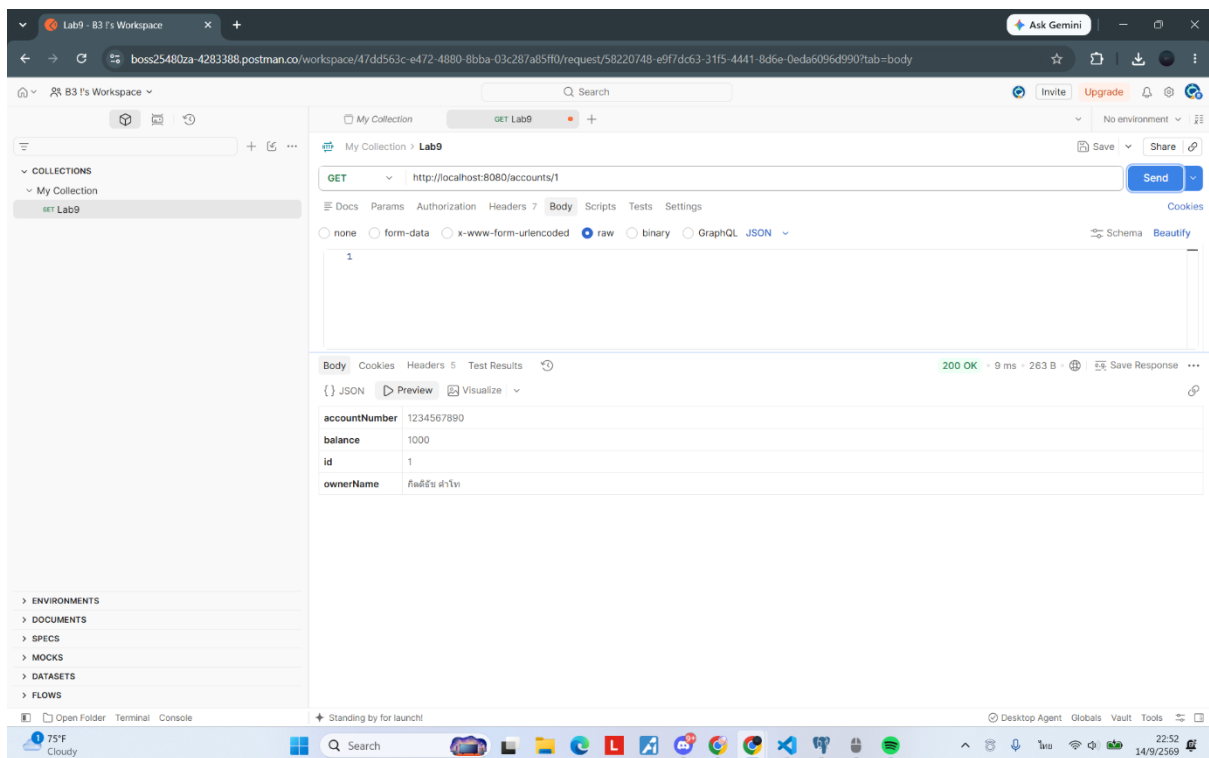
The screenshot shows the Postman interface. The left pane shows the 'Lab9' collection. The main pane shows a POST request to 'http://localhost:8080/accounts/1/deposit' with a body of:

```
{
  "amount": 1000
}
```

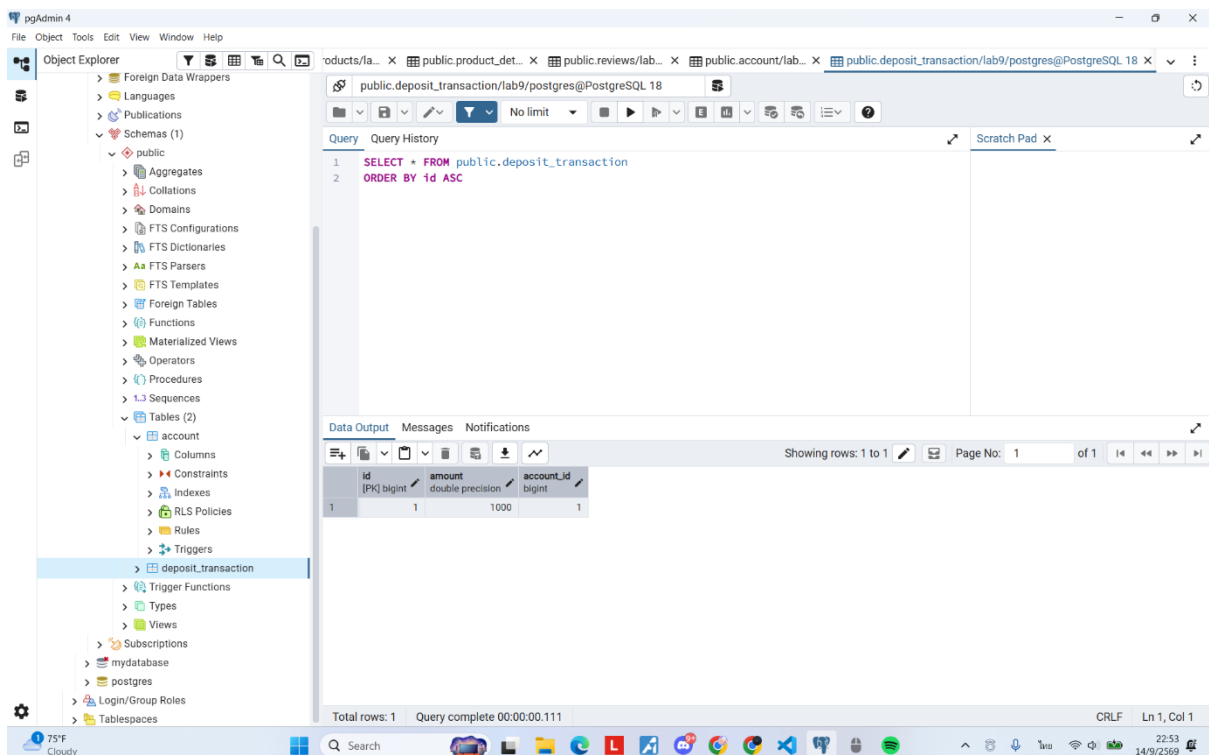
The response pane shows a '500 Internal Server Error' with the following details:

timestamp	status	error	path
2028-09-14T15:56:46.417Z	500	Internal Server Error	/accounts/1/deposit

Postman แสดง Status 500 ตอนยิง deposit เข้าหลังใส่ throw new RuntimeException

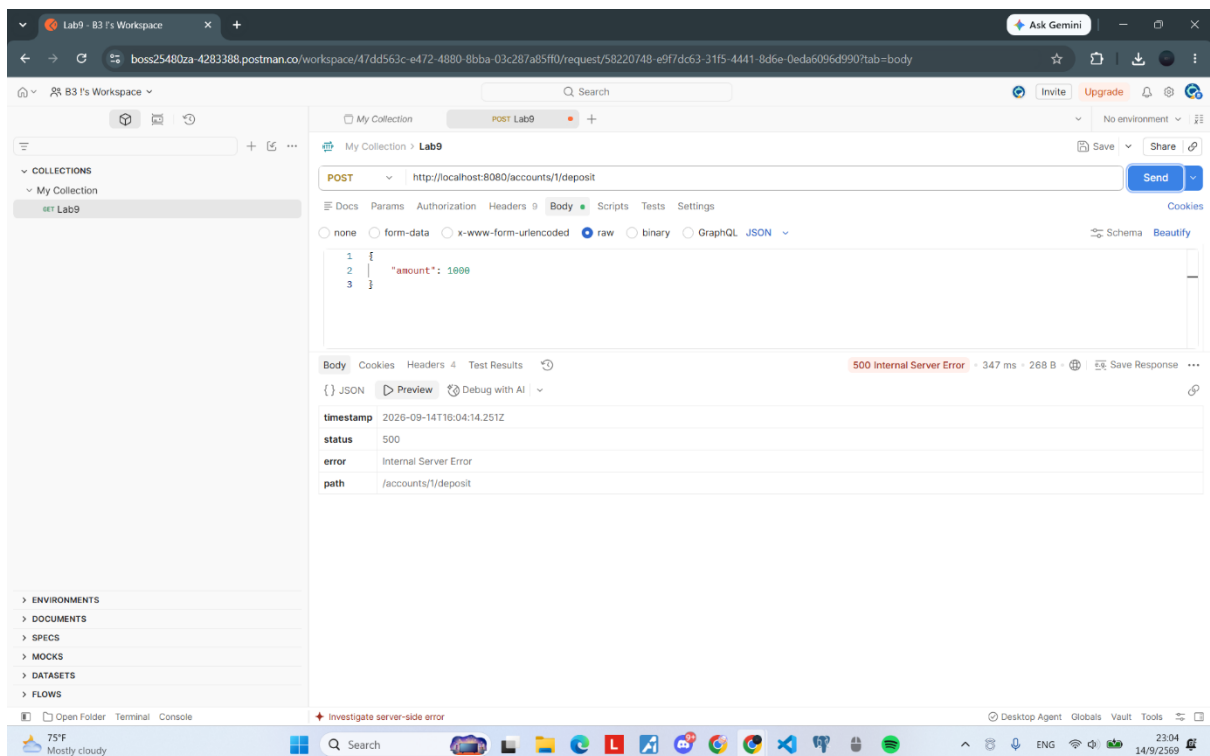


GET account แล้ว balance ยังเหมือนเดิม

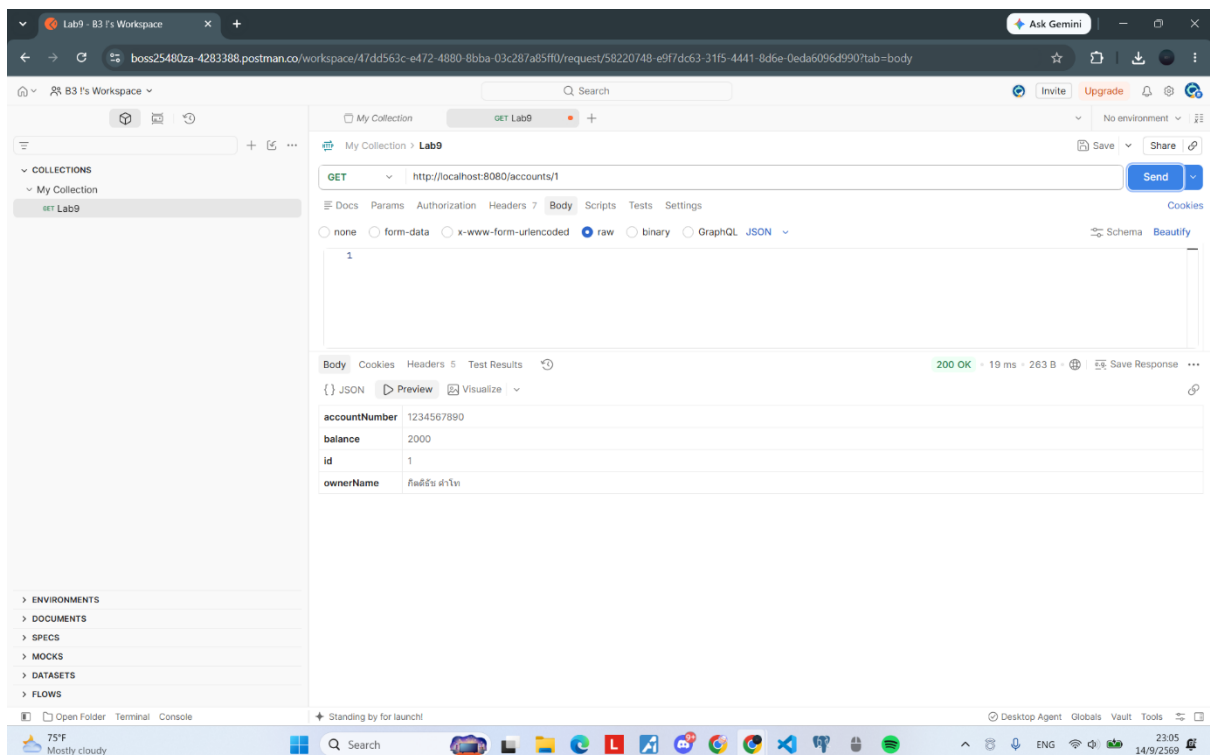


ตาราง deposit_transaction ใน pgAdmin ยังมีแค่แถวเดิม ไม่มีแถวใหม่

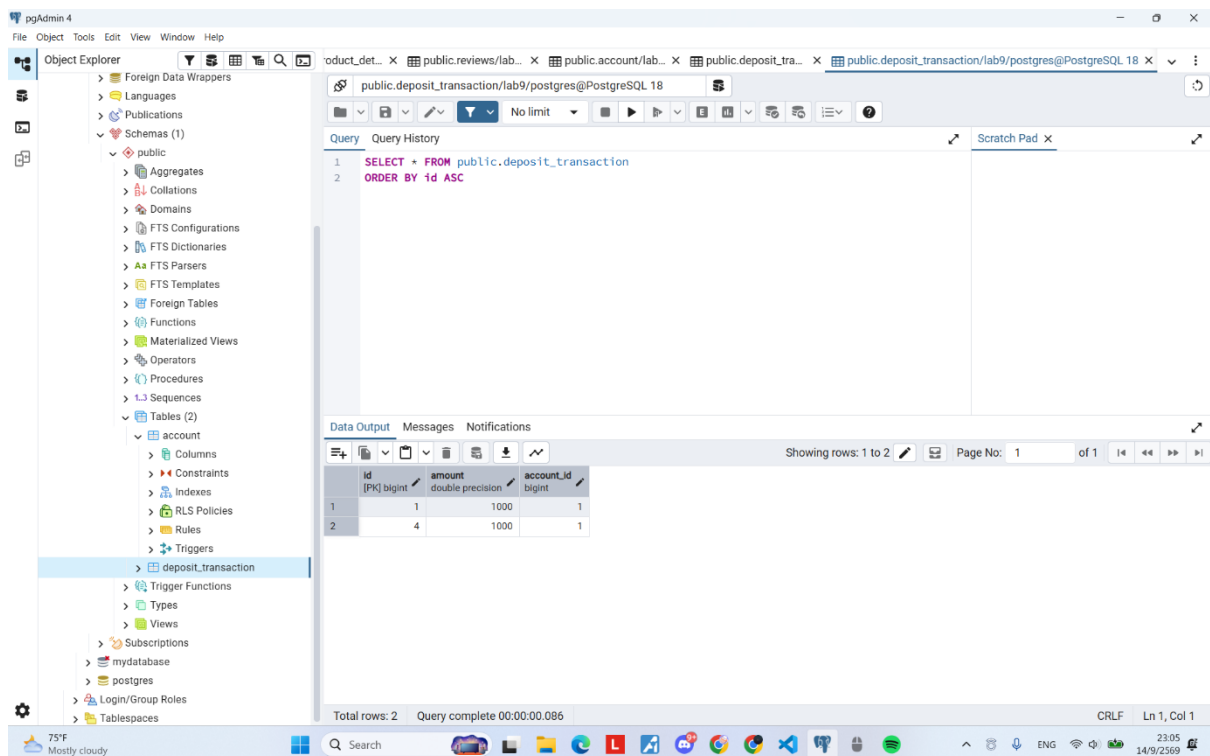
6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional



Postman ยังแสดง Status 500 ตอนยิง deposit



balance เพิ่มขึ้นจริง ทั้งที่เกิต error



มีแถวใหม่เพิ่ม ทั้งที่เกิด error

7. ตอบคำถามท้าย Lab ทั้ง 6 ข้อ

@Transactional มีหน้าที่อะไร?

ทำให้งานเป็นก้อนเดียว ถ้าทุกงานสำเร็จหมดจะบันทึกข้อมูลจริง แต่ถ้าไม่จะ ROLLBACK กับไปทั้งหมด

เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝากเงิน1ครั้งต้องเปลี่ยนแปลงทั้ง2จุด เมื่อฝากเงินไม่สำเร็จก็จะ ROLLBACK กลับไป ทำให้ดีกับระบบฝากเงิน

COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือการยืนยันการบันทึกข้อมูลจริงเมื่อทุกคำสั่งสำเร็จหมด ROLLBACK คือ ยกเลิกการเปลี่ยนแปลงทั้งหมด

Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ONE TO MANY

เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

จะไม่เปลี่ยนแปลง เพราะ @Transactional จะสั่งให้ ROLLBACK

จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

แตกต่างเพราะ ไม่ @Transactional ค่อยสั่งให้ ROLLBACK กลับไป ทำให้ข้อมูลที่ผิดยังคงบันทึกลงข้อมูล